UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

Transformada Lineal de Hough con CUDA

Proyecto No. 3

Curso: Computación Paralela y Distribuida
Sección: 30
Catedrático: Marlón Fuentes

José Ordoñez 231329
Pablo Méndez 23975
Osman De León 23428

Guatemala, noviembre de 2026

Repositorio: `https://github.com/JoseOrdonezB/Proyecto-3-Paralela.git`

Índice

1. Introducción **1**

2. Objetivos **1**
2.1. Objetivo general . . . 1
2.2. Objetivos específicos . . . 1

3. Marco teórico **1**
3.1. Transformada Lineal de Hough . . . 1
3.2. Conversión de coordenadas de imagen . . . 2
3.3. Modelo de ejecución y jerarquía de memoria CUDA . . . 2

4. Diseño e implementación **2**
4.1. Estructura del programa . . . 2
4.2. Versión 1: memoria Global . . . 3
4.3. Versión 2: memoria Global y Constante . . . 3
4.4. Versión 3: memoria Global, Constante y Compartida . . . 3
4.5. Detección de líneas y salida visual . . . 4
4.6. Gestión de recursos . . . 5

5. Metodología experimental **5**
5.1. Entorno de prueba . . . 5
5.2. Medición con eventos CUDA . . . 6
5.3. Validación frente a CPU . . . 6

6. Resultados y análisis **6**
6.1. Comparación principal en RTX 4070 . . . 6
6.2. Interpretación del efecto de la memoria Constante . . . 7
6.3. Interpretación del efecto de la memoria Compartida . . . 7

7. Conclusiones **7**

8. Recomendaciones **8**

9. Referencias **8**

A. Bitácoras de tiempos **8**
A.1. Memoria Global: diez mediciones en RTX 4070 . . . 8
A.2. Memoria Global + Constante: dos series completas en Tesla T4 . . . 9
A.3. Memoria Global + Constante + Compartida: diez mediciones en RTX 4070 . . . 9

B. Detalle de la validación numérica **10**

C. Material suplementario y comandos de reproducción **10**

1. Introducción

La Transformada de Hough permite detectar líneas rectas a partir de los píxeles de una imagen. En vez de buscar directamente cada línea, el algoritmo acumula votos para distintas combinaciones de ángulo y distancia. Este trabajo se puede repartir entre muchos hilos de una GPU, aunque los votos sobre un mismo contador deben coordinarse.

En el proyecto se desarrollaron tres versiones CUDA: una con memoria Global, otra que agrega memoria Constante para senos y cosenos, y una tercera que usa memoria Compartida para acumular votos por bloque. Se compararon los resultados con una versión secuencial, se midieron los kernels con CUDA events y se generaron imágenes con las líneas detectadas. El propósito fue observar qué cambia al usar cada tipo de memoria y qué ventajas se obtienen en una prueba real.

2. Objetivos

2.1. Objetivo general

Implementar y evaluar la Transformada Lineal de Hough en CUDA mediante tres configuraciones de memoria GPU, comprobando su corrección frente a CPU y comparando los tiempos de ejecución de los kernels.

2.2. Objetivos específicos

- Implementar la transformación usando únicamente memoria Global como referencia paralela.
- Incorporar memoria Constante para almacenar senos y cosenos precalculados.
- Diseñar un acumulador local por bloque en memoria Compartida, con sincronización y operaciones atómicas.
- Medir al menos diez ejecuciones de cada variante mediante CUDA events.
- Validar el acumulador GPU contra la implementación secuencial y visualizar las líneas que superan un umbral.
- Analizar la aceleración relativa y las limitaciones de los resultados experimentales.

3. Marco teórico

3.1. Transformada Lineal de Hough

Una línea recta puede representarse en forma normal por

$$r(\theta) = x\cos(\theta) + y\sin(\theta), \tag{1}$$

donde r es la distancia con signo desde el origen hasta la línea y θ es el ángulo de su vector normal [4, 5]. Esta representación evita la singularidad de las líneas verticales presente en la forma pendiente–intercepto. Cada píxel activo (x, y) aporta votos a una familia de pares (r, θ); un máximo local del acumulador indica una línea respaldada por numerosos píxeles.

La implementación del proyecto discretiza el ángulo en 90 intervalos de dos grados y la distancia en 100 contenedores. Por tanto, el acumulador posee $90 \times 100 = 9000$ contadores. Para una imagen de ancho w y alto h, se utiliza el centro como origen y una distancia máxima aproximada de

$$r_{\text{máx}} = \frac{\sqrt{w^2 + h^2}}{2}, \qquad \Delta r = \frac{2r_{\text{máx}}}{100}. \tag{2}$$

El índice discreto de distancia se obtiene desplazando el rango negativo y escalándolo:

$$rIdx = \left\lfloor \frac{r + r_{\text{máx}}}{\Delta r} \right\rfloor, \qquad 0 \leq rIdx < 100. \tag{3}$$

El índice lineal del acumulador corresponde a $rIdx \cdot 90 + \text{thetaIdx}$.

3.2. Conversión de coordenadas de imagen

Una imagen PGM se almacena por filas; el índice lineal de un píxel se expresa como $idx = j \cdot w + i$. Las coordenadas cartesianas usadas en la transformación se calculan como

$$xCoord = i - \left\lfloor \frac{w}{2} \right\rfloor, \qquad yCoord = \left\lfloor \frac{h}{2} \right\rfloor - j. \tag{4}$$

La primera expresión traslada el origen horizontal al centro. La segunda también centra el origen, pero invierte el eje vertical: en una imagen las filas crecen hacia abajo, mientras que en el plano cartesiano el eje y positivo crece hacia arriba. Así, la ecuación (1) utiliza un sistema de referencia coherente.

3.3. Modelo de ejecución y jerarquía de memoria CUDA

CUDA agrupa hilos en bloques y bloques en un grid. Cada hilo obtiene un identificador global para determinar qué píxel procesará. La memoria Global es visible para los hilos del dispositivo y permite almacenar la imagen y el acumulador final; sin embargo, las escrituras concurrentes requieren coordinación. La memoria Constante es de solo lectura desde el kernel y está optimizada para accesos uniformes de los hilos de un warp. La memoria Compartida reside a nivel de bloque y permite cooperación rápida entre sus hilos, aunque su capacidad por bloque es limitada [2].

Las barreras `__syncthreads()` esperan a los hilos del bloque y ordenan las etapas de trabajo. Las operaciones `atomicAdd` garantizan que incrementos concurrentes no pierdan actualizaciones. En este problema son esenciales porque varios píxeles pueden votar por el mismo bin.

4. Diseño e implementación

4.1. Estructura del programa

El proyecto utiliza tres programas CUDA independientes, una imagen de entrada `runway.pgm` y utilidades auxiliares de lectura PGM y dibujo de líneas. El archivo `Makefile` contiene los objetivos de compilación. En cada variante se inicializan los datos, se ejecuta el kernel, se recupera el acumulador y se contrasta con la referencia CPU. Las variantes Constante y Compartida añaden la detección de líneas y la escritura de PNG.

4.2. Versión 1: memoria Global

La versión `houghBase.cu` asigna en memoria Global los datos de entrada, los arreglos trigonométricos y el acumulador de votos. Cada hilo procesa un píxel, calcula sus coordenadas centradas y, si el píxel está activo, recorre los 90 ángulos. Los incrementos del acumulador se realizan con operaciones atómicas para impedir condiciones de carrera.

Para una geometría unidimensional de bloques y grid, el identificador global se calcula mediante

$$gloID = blockIdx.x \cdot blockDim.x + threadIdx.x. \tag{5}$$

La configuración del grid debe cubrir $w \cdot h$ píxeles, normalmente con división entera redondeada hacia arriba. Los hilos sobrantes no deben acceder fuera de la imagen. Esta variante sirve como línea base para medir el efecto de las memorias adicionales.

4.3. Versión 2: memoria Global y Constante

La versión `houghConst.cu` declara `d_Cos` y `d_Sin` como arreglos `__constant__` de 90 elementos. Los valores se calculan previamente en CPU y se transfieren al dispositivo con `cudaMemcpyToSymbol`. Al ser símbolos globales del dispositivo, no es necesario pasar punteros trigonométricos como parámetros del kernel. Los píxeles y el acumulador continúan en memoria Global.

Esta decisión aprovecha que los 90 ángulos son idénticos para todos los píxeles y los valores trigonométricos no cambian durante el kernel. La memoria Constante puede distribuir eficazmente una misma lectura a múltiples hilos de un warp; no obstante, la mejora depende de la configuración y de la importancia relativa del costo de los accesos y de las operaciones atómicas [2].

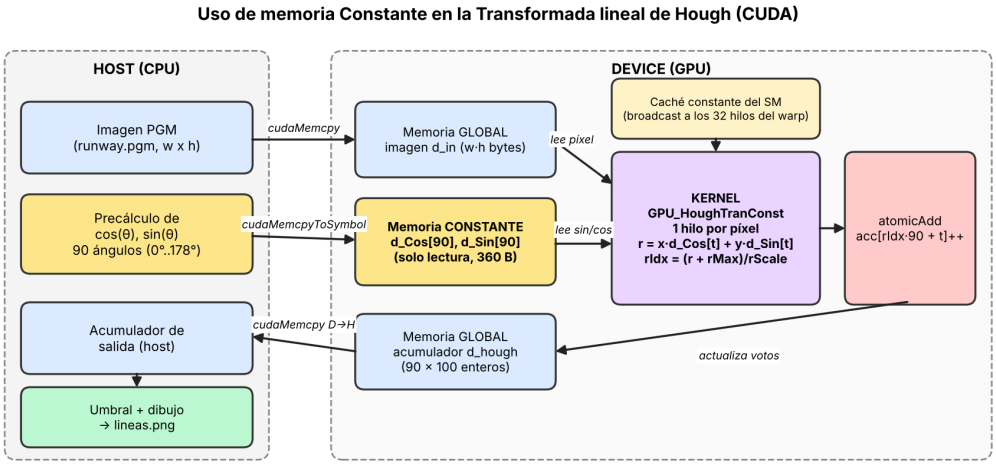


Figura 1: Diagrama funcional de la memoria Constante: transferencia de valores trigonométricos y consulta desde el kernel. Fuente: elaboración del equipo.

4.4. Versión 3: memoria Global, Constante y Compartida

La versión `houghShared.cu` conserva la memoria Constante y añade un acumulador por bloque denominado `localAcc`. Tiene $90 \cdot 100 = 9000$ enteros de cuatro bytes, equivalentes a 36 000 bytes (aproximadamente 35.2 KiB) de memoria Compartida por bloque. La implementación utiliza un

índice local `locID`, derivado de los identificadores de hilos del bloque, para distribuir la inicialización y la reducción del acumulador.

La secuencia es la siguiente:

1. Los hilos inicializan cooperativamente los 9000 contadores de `localAcc` en cero.
2. Una primera barrera `__syncthreads()` impide comenzar la votación antes de completar la inicialización.
3. Cada hilo válido evalúa su píxel y suma votos mediante `atomicAdd` sobre el acumulador local.
4. Una segunda barrera garantiza que todos los votos locales estén completos.
5. Los hilos recorren los bins asignados y agregan los valores locales al acumulador Global mediante operaciones atómicas.

Es importante que los hilos que queden fuera de los límites de la imagen también participen en las barreras; no deben abandonar anticipadamente el kernel antes de la sincronización. El esquema reduce el número de actualizaciones directas al acumulador Global, a cambio de inicializar y reducir un acumulador por bloque.

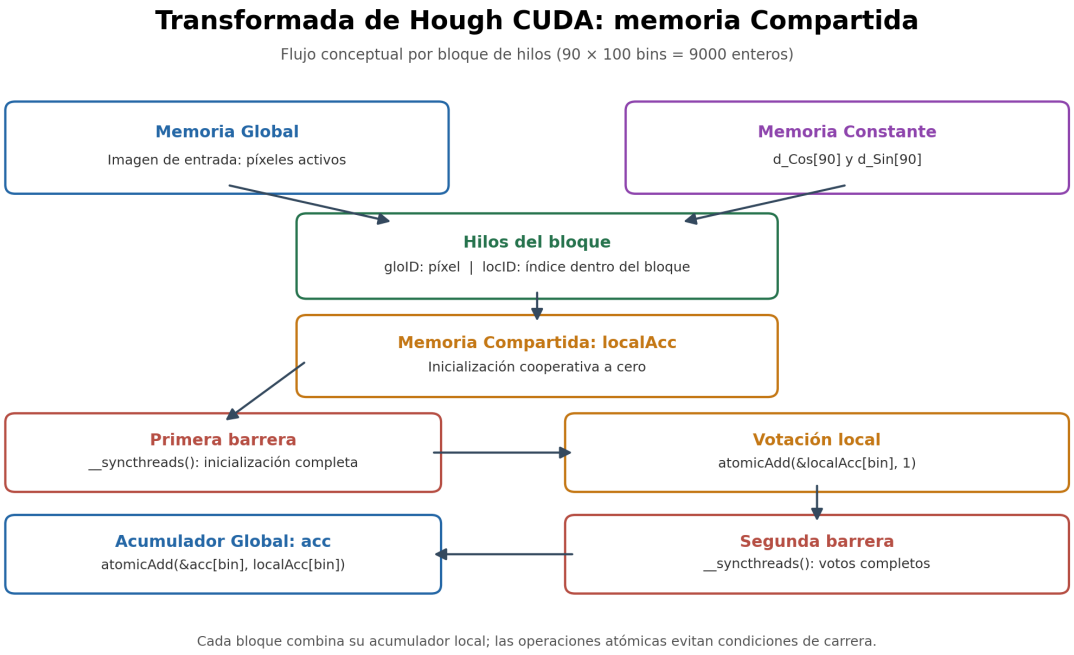


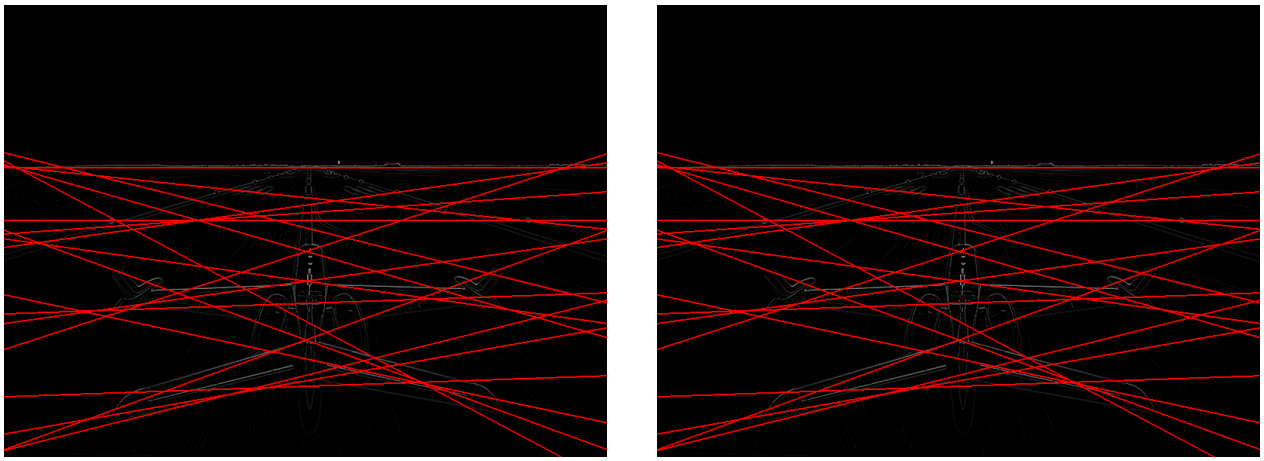
Figura 2: Diagrama funcional de memoria Compartida, barreras y suma atómica hacia el acumulador Global. Fuente: elaboración del equipo.

4.5. Detección de líneas y salida visual

Tras recuperar el acumulador, las versiones con visualización calculan su media μ y desviación estándar σ. Se utiliza el criterio

$$T = \mu + k\sigma. \tag{6}$$

Para $k = 2$, la ejecución reportó $\mu = 1384{,}06$, $\sigma = 1078{,}75$ y $T = 3541{,}55$ votos. Los bins cuyo peso supera el umbral se interpretan como líneas candidatas y se dibujan en rojo sobre la imagen original. Con la entrada de prueba se detectaron 18 líneas. Este umbral es heurístico y su valor afecta la cantidad de líneas mostradas; no equivale por sí solo a una clasificación semántica de los objetos de la escena.



(a) Global + Constante, $k = 2$. **(b)** Global + Constante + Compartida, $k = 2$.

Figura 3: Líneas detectadas superpuestas en rojo sobre la imagen de entrada. Fuente: resultados del proyecto.

4.6. Gestión de recursos

Las implementaciones reservan memoria de dispositivo para la imagen y los acumuladores, y liberan las asignaciones mediante `cudaFree`. Los eventos utilizados para medición se destruyen con `cudaEventDestroy`; los recursos de host también deben liberarse al terminar. El proyecto contempla el reinicio de recursos del dispositivo. Esta gestión evita fugas entre ejecuciones y responde a los requisitos de la rúbrica.

5. Metodología experimental

5.1. Entorno de prueba

Las mediciones comparativas principales se realizaron en una NVIDIA GeForce RTX 4070 con Ubuntu 24.04.4 LTS sobre WSL2, usando la imagen `runway.pgm` de 800×600 píxeles. Cada variante se midió diez veces. Se mantuvo el mismo tamaño de entrada y la misma discretización de 90 ángulos y 100 distancias. Se reportan tiempos del *kernel*, no el tiempo total de lectura de imagen, transferencias y generación de PNG.

La bitácora de memoria Constante también conserva dos series previas de diez mediciones realizadas con una NVIDIA Tesla T4 en Google Colab. Esas series se incluyen como evidencia adicional, pero no se mezclan con las mediciones de la RTX 4070 para calcular aceleraciones.

5.2. Medición con eventos CUDA

Para medir la duración del kernel se crean eventos CUDA, se registra un evento antes del lanzamiento y otro después, se sincroniza el evento final y se consulta el tiempo transcurrido mediante `cudaEventElapsedTime`, que devuelve milisegundos [3]. La sincronización es necesaria porque el lanzamiento de kernels es asíncrono respecto al host. Se repite la medición y se calculan media y dispersión; los resultados deben interpretarse considerando la carga y las frecuencias de la GPU.

5.3. Validación frente a CPU

El acumulador GPU se compara bin por bin con el de la implementación secuencial. En las pruebas documentadas de Global y Compartida coincidieron exactamente 8994 de 9000 bins; los otros seis difirieron en un voto. En la ejecución Compartida se reportaron 12 456 540 votos tanto para CPU como para GPU. La validación se considera aceptada *con tolerancia*, no idéntica bit a bit. Las discrepancias son compatibles con redondeos de punto flotante y la discretización de r, pero su causa exacta no fue aislada experimentalmente.

6. Resultados y análisis

6.1. Comparación principal en RTX 4070

Tabla 1: Tiempo promedio de kernel por estrategia de memoria en RTX 4070.

Versión	n	Promedio (ms)	Aceleración vs. Global
Global	10	0.880922	1.000×
Global + Constante	10	0.877856	1.003×
Global + Constante + Compartida	10	**0.365782**	**2.408**×

La aceleración se calcula como $S = T_{Global}/T_{variante}$. El tiempo de Global + Constante fue aproximadamente 0.35 % menor que el de Global, una diferencia pequeña frente a la variabilidad normal de las ejecuciones. No se puede afirmar con estos datos que el uso de memoria Constante produzca una mejora estadísticamente concluyente. En cambio, la versión que incorpora memoria Compartida redujo el tiempo medio aproximadamente 58.5 % frente a Global, equivalente a una aceleración de 2.41×.

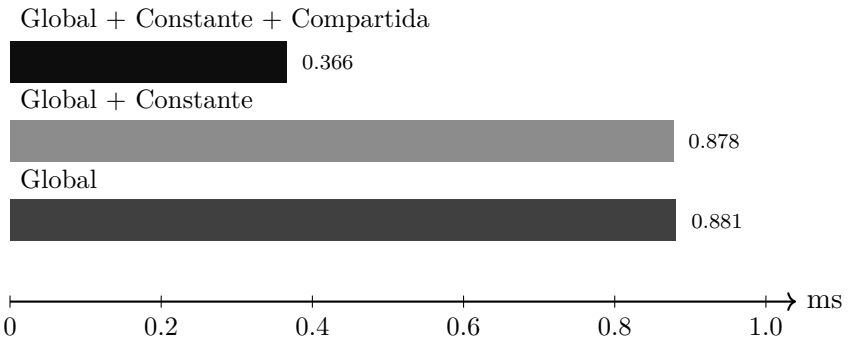


Figura 4: Comparación de los promedios de kernel en la RTX 4070. Fuente: mediciones del proyecto.

6.2. Interpretación del efecto de la memoria Constante

La ventaja potencial de memoria Constante se debe a la reutilización de los mismos valores trigonométricos por numerosos hilos. Sin embargo, en esta carga el tiempo total también incluye cálculos de coordenadas y operaciones atómicas sobre el acumulador. Además, las GPU modernas poseen jerarquías de caché que pueden servir eficientemente datos de lectura repetida. Por ello, la pequeña diferencia observada entre Global y Global + Constante no es inesperada y no implica que la memoria Constante sea inútil: simplemente no dominó el costo medido en esta configuración.

6.3. Interpretación del efecto de la memoria Compartida

La versión Compartida agrupa las votaciones por bloque antes de sumarlas al acumulador Global. Esta estrategia puede reducir la presión de escrituras atómicas globales cuando muchos píxeles votan por los mismos bins. Su costo adicional consiste en inicializar los 9000 contadores locales, sincronizar los hilos y fusionar los resultados. En el entorno evaluado, el beneficio neto fue considerable. El resultado no garantiza la misma aceleración para cualquier imagen o GPU: la densidad de píxeles activos, la ocupación y el tamaño de memoria Compartida por bloque pueden cambiar el balance.

7. Conclusiones

La versión con memoria Compartida fue la más rápida en la RTX 4070: pasó de 0.880922 ms (Global) a 0.365782 ms, una aceleración de 2.41 veces. La memoria Constante, por sí sola, apenas cambió el promedio (0.877856 ms), por lo que no sería correcto afirmar que siempre mejora el rendimiento.

La acumulación local por bloque ayudó a reducir las actualizaciones directas a memoria Global. Para que esto funcione, las operaciones atómicas y las dos barreras de sincronización son indispensables. En la validación, 8994 de 9000 bins coincidieron con CPU; los otros seis difirieron en un voto. Por ello, los resultados se consideran correctos con la tolerancia indicada, no idénticos.

Con un umbral de media más dos desviaciones estándar se dibujaron 18 líneas sobre la imagen de prueba, lo que permitió verificar visualmente la salida del algoritmo.

8. Recomendaciones

Conviene repetir las pruebas con más imágenes y tamaños de bloque, y conservar los tiempos individuales de las diez corridas de Constante en la RTX 4070. También sería útil estudiar las seis diferencias de redondeo y medir el tiempo total del programa, además del kernel. Las comparaciones entre Tesla T4 y RTX 4070 deben mantenerse separadas.

9. Referencias

Referencias

[1] Universidad del Valle de Guatemala. *Proyecto 3: Transformada Lineal de Hough en CUDA*. Enunciado proporcionado para Computación Paralela y Distribuida.

[2] NVIDIA. *CUDA C++ Programming Guide*. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`.

[3] NVIDIA Developer Blog. *How to Implement Performance Metrics in CUDA C/C++*. `https://developer.nvidia.com/blog/how-implement-performance-metrics-cuda-cc/`.

[4] Duda, R. O. y Hart, P. E. (1972). *Use of the Hough Transformation to Detect Lines and Curves in Pictures*. Communications of the ACM, 15(1), 11–15.

[5] Hough, P. V. C. (1962). *Method and Means for Recognizing Complex Patterns*. Patente estadounidense US3069654A.

A. Bitácoras de tiempos

A.1. Memoria Global: diez mediciones en RTX 4070

Tabla 2: Diez mediciones individuales de la versión Global.

Corrida	Tiempo (ms)	Corrida	Tiempo (ms)
1	0.880640	6	0.883712
2	0.881664	7	0.883712
3	0.898816	8	0.878592
4	0.879616	9	0.873472
5	0.872448	10	0.876544

Promedio reportado: **0.880922 ms**; mínimo: 0.872448 ms; máximo: 0.898816 ms.

A.2. Memoria Global + Constante: dos series completas en Tesla T4

Tabla 3: Mediciones originales en Tesla T4, conservadas como evidencia independiente.

Corrida	Serie A (ms)	Serie B (ms)
1	1.339392	1.545952
2	1.318976	1.555136
3	1.327648	1.538304
4	1.332992	1.534912
5	1.337344	1.537440
6	1.314848	1.552384
7	1.328064	1.539616
8	1.336960	1.547200
9	1.320896	1.541856
10	1.335328	1.525888
Promedio	1.329245	1.541869

En RTX 4070 se registraron **10 corridas**, con promedio **0.877856 ms** y desviación estándar **0.005083 ms**. La salida disponible de esa prueba no conserva los diez valores individuales; por transparencia, no se reconstruyen ni se inventan.

A.3. Memoria Global + Constante + Compartida: diez mediciones en RTX 4070

Tabla 4: Diez mediciones individuales de la versión Compartida.

Corrida	Tiempo (ms)	Corrida	Tiempo (ms)
1	0.367616	6	0.363520
2	0.363520	7	0.382976
3	0.363520	8	0.363520
4	0.363520	9	0.363520
5	0.362496	10	0.363616

Promedio reportado: **0.365782 ms**; desviación estándar: **0.006193 ms**; mínimo: 0.362496 ms; máximo: 0.382976 ms.

B. Detalle de la validación numérica

Tabla 5: Seis bins con diferencia unitaria en la comparación CPU/GPU.

Bin	CPU	GPU
1803	1446	1445
1893	1506	1507
5931	1653	1654
6021	1816	1815
6104	1642	1641
6194	1586	1587

Los restantes 8994 bins coincidieron exactamente. El acumulador de la versión Compartida produjo 12 456 540 votos totales, igual que la referencia CPU.

C. Material suplementario y comandos de reproducción

```
make hough
make houghConst
make houghShared
./hough runway.pgm
./houghConst runway.pgm 10 2.0 resultados/lineas_constante.png
./houghShared runway.pgm 10 2.0 lineas_shared.png
```

El código fuente, las bitácoras y los diagramas se conservan en el repositorio del proyecto: `https://github.com/JoseOrdonezB/Proyecto-3-Paralela.git`. Para reproducir los resultados deben utilizarse una GPU NVIDIA compatible, el compilador `nvcc` y el archivo de entrada indicado.